

Tab 1

Time-series template

1. Imports (ALWAYS START HERE)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
pd.set_option('display.max_columns', None)
```

2. Load Dataset

```
# Load CSV / TSV / Excel
```

```
df = pd.read_csv("data.csv")          # CSV
```

```
# df = pd.read_csv("data.tsv", sep="\t") # TSV
```

```
# df = pd.read_excel("data.xlsx")      # Excel
```

```
print(df.shape)
```

```
df.head()
```

3. Basic Data Understanding

```
df.info()
```

```
df.describe(include="all")
```

✓ Shows:

Data types

Missing values

Numeric stats

Categorical cardinality

4. Missing Value Analysis & Cleaning

```
# % rows with missing values
```

```
missing_rows = df.isnull().any(axis=1).sum()
```

```
missing_pct = (missing_rows / len(df)) * 100
```

```
print(f'Missing rows: {missing_pct:.2f}%')
```

```
# Option 1: Drop rows
```

```
df = df.dropna()
```

```
# Option 2: Impute (use when required)
# df['numeric_col'] = df['numeric_col'].fillna(df['numeric_col'].mean())
# df['cat_col'] = df['cat_col'].fillna(df['cat_col'].mode()[0])
```

5. Data Cleaning (EXAM GOLD SECTION ★)

```
# Standardise text columns
cat_cols = df.select_dtypes(include='object').columns
```

```
for col in cat_cols:
    df[col] = df[col].str.lower().str.strip()
```

6. Irregular Cardinality Handling

```
# Identify high-cardinality features
```

```
for col in cat_cols:
    print(col, df[col].nunique())
```

```
# Group rare categories
value_counts = df['category_col'].value_counts()
rare_values = value_counts[value_counts < 10].index
df['category_col'] = df['category_col'].replace(rare_values, 'other')
```

💡 Exam phrase to remember:

“Rare categories were grouped to reduce noise and overfitting.”

7. Feature Engineering (VERY IMPORTANT)

```
# Date features
```

```
df['date'] = pd.to_datetime(df['date'])
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
```

```
# Binning
```

```
df['age_group'] = pd.cut(df['age'], bins=[0,5,10,20,50])
```

```
# Log transform (skewed data)
```

```
df['log_price'] = np.log1p(df['price'])
```

8. Exploratory Data Analysis (EDA)

Numeric vs Numeric

```
plt.figure()
plt.scatter(df['x'], df['y'])
plt.xlabel("X")
```

```
plt.ylabel("Y")
plt.show()
```

Categorical vs Numeric

```
plt.figure()
sns.boxplot(x='category', y='value', data=df)
plt.show()
```

Categorical vs Categorical

```
ct = pd.crosstab(df['cat1'], df['cat2'])
```

```
plt.figure(figsize=(8,5))
sns.heatmap(ct, annot=True, fmt='d')
plt.show()
```

Distribution Plot

```
plt.figure()
plt.hist(df['numeric_feature'], bins=20)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

9. Filtering & Subsetting

```
filtered_df = df[df['region'].isin(['a', 'b'])]
percentage_retained = len(filtered_df) / len(df) * 100
print(f"Rows retained: {percentage_retained:.2f}%")
```

10. GroupBy & Aggregation

```
grouped = (
    df.groupby('category')['value']
    .agg(['mean', 'sum', 'count'])
)
```

grouped

11. Time Series Analysis (COMMON EXAM TASK)

```
df['date'] = pd.to_datetime(df['date'])
df.set_index('date', inplace=True)
```

```
monthly = df.resample('M').size()
```

```
plt.figure()
monthly.plot()
plt.xlabel("Time")
```

```
plt.ylabel("Count")
plt.show()
```

12. Multiple Time Series on One Plot

```
ts = (
    df.groupby([pd.Grouper(freq='M'), 'fuel_type'])
        .size()
        .unstack(fill_value=0)
)
```

```
plt.figure(figsize=(10,6))
ts.plot()
plt.show()
```

13. Encoding (If ML is Required)

One-hot encoding

```
df_encoded = pd.get_dummies(df, drop_first=True)
```

14. Scaling (If Required)

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df_encoded)
```

Steps to Build a Classifier (Coding Workflow Order)

1. Import required libraries

- pandas, numpy
- sklearn preprocessing, models, metrics

2. Load the dataset

- Read CSV / TSV into a DataFrame

3. Initial data inspection

- Check shape, columns, data types
- Identify missing values and class distribution

4. Separate features and target

- X = input features
- y = class labels

5. Encode the target variable (if needed)

- Convert categorical labels to numeric

6. Identify feature types

- Numerical features
- Categorical features

7. Split data into training and test sets

- Use stratified split for classification

8. Preprocess features

- Handle missing values
- Encode categorical features
- Scale / normalize numerical features

9. Build preprocessing + model pipeline

- Combine preprocessing and classifier

10. Choose the classification algorithm

- Logistic Regression / Naive Bayes / Decision Tree / etc.

11. Train the model

- Fit pipeline on training data

12. Test the model

- Predict on test data

13. Evaluate model performance

- Accuracy, Precision, Recall, F1
- Confusion matrix, ROC-AUC

14. Visualise results (optional)

- Confusion matrix plot
- ROC curve

15. Hyperparameter tuning (optional)

- Grid search / cross-validation

16. Final model selection

- Choose best performing model

by vamsi



Python Data Science Practical Exam – Ready-to-Paste Templates



TEMPLATE 0: Notebook Setup (ALWAYS FIRST CELL)

```
# === IMPORTS ===  
# Do NOT remove unless explicitly unnecessary  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression, LogisticRegression  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import accuracy_score, mean_squared_error  
  
# Make plots display nicely  
plt.rcParams['figure.figsize'] = (8, 5)
```



Change nothing here unless instructed



TEMPLATE 1: Data Loading & Initial Inspection

Markdown Cell

```
## Task X – Data Loading and Inspection  
We begin by loading the dataset provided and inspecting its structure.
```

Code Cell

```
# CHANGE: replace with actual file name provided in exam  
df = pd.read_csv("DATASET_NAME.csv") #CSV  
df = pd.read_csv("data.tsv", sep="\t") # TSV  
df = pd.read_excel("data.xlsx") # Excel  
# Quick inspection  
df.head()
```

```
# Dataset structure and summary  
df.info()
```

```
# Statistical summary (numerical columns only)  
df.describe()
```

```
#Provides min, max, mean, std, and percentiles for numeric columns.  
df.describe(include="all").T
```



Change:

- `DATASET_NAME.csv` → actual dataset name
-



TEMPLATE 2: Data Cleaning & Preprocessing

Q5. Identify numeric and categorical features

```
import numpy as np  
  
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()  
  
cat_cols = df.select_dtypes(exclude=[np.number]).columns.tolist()  
  
print("Numeric:", num_cols)  
  
print("Categorical:", cat_cols)
```

Markdown Cell

```
## Data Cleaning
```

The dataset is cleaned to ensure it is suitable for analysis and modelling.

Code Cells

```
# Check for missing values
```

```
df.isna().sum()
```

```
# OPTION 1: Drop missing values (use if many rows exist)
```

```
df_clean = df.dropna()
```

OPTION 2: Fill missing values (use if dropping is not suitable)

CHANGE: choose appropriate column and strategy

```
df_clean = df.copy()
df_clean['COLUMN_NAME'] =
df_clean['COLUMN_NAME'].fillna(df_clean['COLUMN_NAME'].mean())
```

Verify cleaning

```
df_clean.isna().sum()
```

TO FILL THE MISSING VALUES (use if dropping is not suitable)

If the column is CATEGORICAL / TEXT

(brand, name, type, category, city, make, model)

```
Df['column name'] = df['column name'].fillna(df['column
name'].mode()[0])
```

If the column is NUMERIC

(age, price, mileage, salary, score)

```
df['column name'] = df['column name'].fillna(df['column
name'].median())
```

```
df_clean = df.copy()

cat_cols = ['name', 'name 2', 'name 3']

for col in cat_cols:
    df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])

df_clean.isna().sum()
```

#Report the percentage of rows with missing values and remove these rows from the dataset.

```
missing_percentage = (df.isnull().sum() / len(df)) * 100
print("Percentage of missing values per column:\n",
missing_percentage[missing_percentage > 0])
```

Change: sometimes you only drop based on specific columns:

```
df.dropna(subset=["col1", "col2"])
```

#Remove duplicates

```
duplicates = df.duplicated().sum()
print("Duplicates:", duplicates)
df = df.drop_duplicates()
```

One-line explanation to write

“Numeric features are imputed using the median, while categorical features are imputed using the mode.”

Change:

- **COLUMN_NAME** → column with missing values
 - Choose **only one** strategy (drop OR fill)
-

TEMPLATE 3: Feature Engineering / New Columns

Markdown Cell

Feature Engineering

New features are created to improve analysis.

Code Cell

```
# CHANGE: logic based on dataset
df_clean['new_feature'] = df_clean['COLUMN_A'] / df_clean['COLUMN_B']

df_clean.head()
```

Change:

- **COLUMN_A, COLUMN_B**
 - Formula based on question
-

TEMPLATE 4: Exploratory Data Analysis (EDA)

Markdown Cell

Exploratory Data Analysis

We explore relationships and patterns in the dataset.

Grouping & Aggregation

CHANGE: group column and aggregation column

```
df_clean.groupby('GROUP_COLUMN')['VALUE_COLUMN'].mean()
```

Sorting

CHANGE: column name and order

```
df_clean.sort_values(by='VALUE_COLUMN', ascending=False).head()
```

 **Change:**

- GROUP_COLUMN
 - VALUE_COLUMN
-

TEMPLATE 5: Data Visualization

Markdown Cell

Data Visualization

The following plots visualise key trends in the data.

Bar Plot

CHANGE: columns

```
plt.bar(df_clean['CATEGORY_COLUMN'], df_clean['NUMERIC_COLUMN'])
```

```
plt.xlabel('Category')
```

```
plt.ylabel('Value')
```

```
plt.title('Bar Chart of Category vs Value')
```

```
plt.show()
```

Scatter Plot

CHANGE: columns

```
plt.scatter(df_clean['X_COLUMN'], df_clean['Y_COLUMN'])
```

```
plt.xlabel('X')
```

```
plt.ylabel('Y')
plt.title('Scatter Plot')
plt.show()
```

 **Change:**

- Column names
 - Axis labels & title
-

TEMPLATE 6: Correlation Analysis

Markdown Cell

Correlation Analysis

Correlation is used to quantify relationships between numerical variables.

Code Cell

```
# CHANGE: select relevant numeric columns
df_clean[['COL1', 'COL2', 'COL3']].corr()
```

 **Change:**

- Column list
-

TEMPLATE 7A: Linear Regression (Regression Task)

Markdown Cell

Linear Regression Model

A linear regression model is trained to predict a continuous target variable.

Code Cells

```
# CHANGE: feature columns and target
X = df_clean[['FEATURE_1', 'FEATURE_2']]
y = df_clean['TARGET']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
mse
```



Change:

- FEATURE_1, FEATURE_2
 - TARGET
-



TEMPLATE 7B: Classification (Logistic / Decision Tree)

Markdown Cell

Classification Model

A classification model is trained to predict a categorical outcome.

Code Cells

```
# CHANGE: feature columns and target
```

```
X = df_clean[['FEATURE_1', 'FEATURE_2']]
```

```
y = df_clean['TARGET_CLASS']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# OPTION 1: Logistic Regression
```

```
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train, y_train)
```

```
# OPTION 2: Decision Tree
```

```
# model = DecisionTreeClassifier(random_state=42)
```

```
# model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

accuracy



Change:

- Feature columns
 - Target column
 - Choose ONE model
-



TEMPLATE 8: Final Interpretation (HIGH MARKS)

Markdown Cell

Conclusion

The analysis demonstrates meaningful patterns in the dataset. The trained model achieved reasonable performance, indicating that the selected features are informative. Further improvements could include additional features or alternative models.



FINAL CHECKLIST BEFORE SUBMISSION

- Notebook runs **top to bottom without errors**
 - All plots have **titles and labels**
 - Markdown explanations present in **every task**
 - File saved as **.ipynb**
-



This template is sufficient to score an A+ if used cleanly and correctly.



How to Know WHAT to Change and HOW to Change It (Beginner-Friendly)

Think of the exam as a **fill-in-the-blanks problem**, not coding from scratch.

1 First Thing You Do After Opening the Dataset (MOST IMPORTANT)

After this line:

```
df = pd.read_csv("DATASET_NAME.csv")
```

👉 Run **ONLY** these 3 commands first:

```
df.head()  
df.info()  
df.describe()
```

Why?

These **tell you everything** you need to know:

- Column names
- Data types (number / text)
- Missing values
- What kind of task is possible

📌 You **NEVER** guess column names. You read them here.

2 How to Know What to Put Instead of **COLUMN_NAME**

Example output from **df.info()**:

```
0   age                500 non-null int64  
1  income              480 non-null float64  
2  gender              500 non-null object  
3  purchased           500 non-null int64
```

Now you know:

Column	Meaning	Type
age	number	numeric
income	number	numeric
gender	category	text
purchase d	target	numeric (0/1)

 **Replace template placeholders like this:**

 **Template:**

```
df_clean['COLUMN_NAME']
```

 **Real exam:**

```
df_clean['income']
```

3 How to Handle Missing Values (No Guessing)

Run:

```
df.isna().sum()
```

Example result:

```
age          0
income       20
gender        0
purchased    0
```

Decision rule (easy):

- If missing values < **5–10%** → fill
- If many rows exist → drop

How to fill (safe choice in exam):

```
df_clean['income'] =  
df_clean['income'].fillna(df_clean['income'].mean())
```

● You do NOT need fancy logic for full marks.

4 How to Know What to Group By

Look at column types:

Column type	Use for
object (text)	groupby
numeric	mean / sum

Example:

```
df_clean.groupby('gender')['income'].mean()
```

📌 Rule:

Group by categories, aggregate numbers

5 How to Choose What to Plot (Very Easy Rule)

Bar chart

Use when:

- X = category
- Y = number

```
plt.bar(df_clean['gender'], df_clean['income'])
```

Scatter plot

Use when:

- Both variables are numeric

```
plt.scatter(df_clean['age'], df_clean['income'])
```

NEVER plot:

- text vs text
 - category vs category
-

6 How to Know If It's Regression or Classification

Look at the **TARGET** column:

Target type	Model
Continuous number	Linear Regression
0/1 or Yes/No	Classification

Example:

```
df['purchased'].unique()
```

If output is:

```
[0 1]
```

👉 **Classification**

7 How to Choose Features (FEATURE_1, etc.)

Rule (exam-safe):

- Use **numeric columns**
- Do NOT use target as feature

Example:

```
X = df_clean[['age', 'income']]  
y = df_clean['purchased']
```

 If unsure:

2 features are enough for full marks

8 How to Know Which Model to Pick

Situation	Pick
Continuous target	LinearRegression
Binary target	LogisticRegression
Asked for tree	DecisionTreeClassifier

 Logistic Regression is always safe for binary targets.

9 How Much Markdown Is Enough? (This Matters)

BAD (lose marks):

Here is the code.

GOOD (full marks):

We first inspect the dataset to understand its structure and identify any missing values.

📌 1–2 sentences per section is perfect.

10 Golden Exam Rule (Remember This)

Never invent. Always inspect.

Everything you need comes from:

```
df.head()  
df.info()  
df.describe()
```

✅ Final Reassurance

You do **NOT** need to:

- Write perfect ML
- Optimize models
- Use advanced Python

You just need to:

- ✓ Replace placeholders correctly
- ✓ Follow the structure
- ✓ Explain simply

Aish

TASK 1 — DATA PREPARATION

Task 1: Data Preparation

The objective of this task is to load the dataset, assess its quality, and apply appropriate preprocessing steps to ensure the data is suitable for subsequent analysis.

Dataset Overview

The dataset contains records collected from a data source relevant to the problem domain. Each row represents a single observation described using a combination of numerical and categorical features.

Before analysis, the dataset must be loaded, inspected, and cleaned to ensure reliability and consistency of results.

>>>>CODE

Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

1.1 Loading the Dataset

The dataset is loaded from a local file into a Pandas DataFrame. An initial preview is used to verify that the data has been read correctly.

>>>>CODE

Load Dataset

For CSV:

```
df = pd.read_csv("dataset.csv")
df.head()
```

For TSV:


```
df = pd.read_csv("dataset.tsv", sep="\t")
df.head()
```

For EXCEL:

```
df = pd.read_excel("dataset.xlsx")
df.head()
```

1.2 Initial Data Inspection

An initial inspection is performed to understand the size of the dataset, the available features, and their corresponding data types.

>>>>>CODE

```
print(df.shape)
df.head()
```

>>>>>CODE

`df.info()` - information of dataset

`df.head()` - shows only the 1st 10 table from dataset

`df.describe()`

1.3 Missing Values Analysis

`df.isna().sum()`

1.4 Handling Missing Values

Missing values are handled using appropriate strategies based on the type of features. Numeric features are imputed using the median, while categorical features are imputed using the mode..

OPTION 1: if column / row has to be printed

>>>>>CODE

```
df_clean = df.copy()
```

>>>>CODE

```
cat_cols = df_clean.select_dtypes(include=["object", "category"]).columns
num_cols = df_clean.select_dtypes(include=["number"]).columns
```

```
for col in cat_cols:
    df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])
```

>>>>CODE

```
for col in num_cols:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())
```

OPTION 2: Drop missing values (use if many rows exist)

```
df_clean = df.dropna()
```

1.5 Verification of Cleaning

After handling missing values, the dataset is rechecked to ensure that no missing values remain.

>>>>CODE

```
df_clean.isna().sum().sum()
```

1.6 Feature Cardinality Analysis

Categorical features are examined to identify high or irregular cardinality, which may complicate analysis and interpretation.

>>>>CODE

```
df_clean[cat_cols].nunique()
```

Where necessary, preprocessing steps such as normalisation or category simplifications are applied to address irregular cardinality.

Task 1 Summary

In this task, the dataset was successfully loaded and inspected. Missing values were handled using appropriate imputation strategies, and

categorical features were examined for irregular cardinality.

The cleaned dataset is suitable for subsequent analysis.

Task 2: Feature Exploration

Task 2: Feature Exploration

The objective of this task is to explore relationships and patterns within the dataset using appropriate visualisations and summary statistics.

>>>

2.1 Relationship Between a Categorical Feature and a Numerical Feature

The relationship between a categorical feature and a numerical feature is examined by aggregating values and visualising the results.

What this does

- *Introduces categorical vs numerical analysis*
- *Very common exam requirement*
- *Keeps it dataset-independent*

>>> Aggregation code

```
grouped_data = (  
    df_clean  
    .groupby("CATEGORICAL_COLUMN")["NUMERICAL_COLUMN"]  
    .sum()  
    .sort_values(ascending=False)  
)
```

What this does

- *Groups data by a categorical feature*
- *Aggregates a numerical feature (sum / mean / count)*

- **Reveals:**
 - *Which category contributes most*
 - *Differences across groups*

 **Replace:**

- **CATEGORICAL_COLUMN** → e.g. *region, category, type*
- **NUMERICAL_COLUMN** → e.g. *price, sales, amount*

>>> PLOT CODE

```
grouped_data.plot(kind="bar", figsize=(8, 5))
plt.xlabel("Category")
plt.ylabel("Aggregated Value")
plt.title("Relationship Between Category and Numerical Feature")
plt.show()
```

(grouped data should have a name of relation of each category)

 **What this does**

- *Visualises the aggregated results*
- *Bar plot is ideal for:*
 - *Category comparisons*
 - *Discrete groups*
- *Makes patterns easy to interpret*

>>>markdown

This plot highlights differences in the numerical feature across categories, allowing comparison of relative contributions between groups.

2.2 Relationship Between Two Categorical Features

The relationship between two categorical features is examined to identify common combinations and potential associations.

>>>Crosstab code

```
cross_tab = pd.crosstab(  
    df_clean["CATEGORICAL_COLUMN_1"],  
    df_clean["CATEGORICAL_COLUMN_2"]  
)
```

cross_tab

👉 *What this does*

- *Counts how often category combinations occur*
- *Helps identify:*
 - *Strong associations*
 - *Dominant combinations*

>>>markdown

The cross-tabulation shows how frequently combinations of the two categorical features occur, helping to identify dominant or rare pairings.

2.3 Distribution of a Numerical Feature

The distribution of a numerical feature is examined to understand its spread, central tendency, and potential skewness.

>>>histogram code

```
plt.figure(figsize=(8, 5))  
plt.hist(df_clean["NUMERICAL_COLUMN"], bins=20)  
plt.xlabel("Value")  
plt.ylabel("Frequency")  
plt.title("Distribution of Numerical Feature")  
plt.show()
```

👉 ***What this does***

- ***Shows how values are distributed***
- ***Helps identify:***
 - ***Skewness***
 - ***Outliers***
 - ***Typical ranges***

>>>markdown

The distribution shows how values are spread across the dataset, indicating whether the feature is skewed or concentrated around specific ranges.

Task 2 Summary

In this task, relationships between categorical and numerical features were explored using aggregation and visualisation. Associations between categorical features were examined, and the distribution of a numerical feature was analysed

to gain insight into the dataset structure.

Task 3

Time Series Analysis

Task 3: Time Series Analysis

The objective of this task is to analyse temporal patterns in the dataset by examining how observations change over time.

3.1 Data Filtering Based on a Condition

The dataset is filtered to include only observations that meet a specific condition, allowing focused time series analysis.

>>> Filtering the Dataset code

```
df_filtered = df_clean[
    df_clean["CATEGORICAL_COLUMN"].isin(["VALUE_1", "VALUE_2"])
]
```

👉 *What this does*

- *Selects only relevant rows*
- *Keeps analysis focused and meaningful*

 **Example replacements:**

- **CATEGORICAL_COLUMN** → *region, category, type*
- **VALUE_1, VALUE_2** → *specific groups of interest*

>>> markdown

The proportion of data retained after filtering is calculated to understand the impact of the filtering step.

>>> Percentage Retained Code

```
percentage_retained = (len(df_filtered) / len(df_clean)) * 100
```

Percentage_retained

 **What this does**

- *Quantifies how much data remains*
- *Shows transparency in preprocessing*

3.2 Time Series Construction

Time series are constructed by aggregating observations over time to examine temporal trends.

>>> **Ensure Date Format code**

```
df_filtered["DATE_COLUMN"] = pd.to_datetime(df_filtered["DATE_COLUMN"])
```


👉 *What this does*

- *Converts column to datetime*
- *Required for resampling and time grouping*

>>> Create Time Series CODE

```
time_series = (  
    df_filtered  
    .groupby(  
        [pd.Grouper(key="DATE_COLUMN", freq="M"), "CATEGORICAL_COLUMN"]  
    )  
    .size()  
    .unstack()  
)
```

👉 *What this does*

- Groups data by:
 - time period (monthly)
 - category
- Produces a table suitable for plotting
- Each column becomes a separate time series

3.3 Visualisation of Time Series

The constructed time series are visualised on a single plot to allow comparison of trends across categories.

```
>>>Plot Time Series  
  
time_series.plot(figsize=(10, 5))  
  
plt.xlabel("Time")  
  
plt.ylabel("Count")  
  
plt.title("Time Series of Observations by Category")  
  
plt.show()
```

👉 What this does

Plots multiple time series together

Makes trend comparison easy

Clearly shows increases, decreases, or stability

>>> markdown

The plot reveals temporal patterns in the data, highlighting differences in trends between categories over time.

Task 3 Summary

In this task, the dataset was filtered to focus on relevant observations.

Time series were constructed by aggregating data over time, and trends were visualised to compare temporal patterns across categories.

◆ Task 1 – Data Preparation

This matches:

- Loading the dataset
- Initial inspection (**shape**, **info**)
- Missing values analysis
- Cleaning (drop OR impute)
- Feature cardinality analysis

👉 This is exactly what you implemented in Task 1.

◆ Task 2 – Feature Exploration

This includes:

- Relationship between categorical ↔ numerical features (e.g., aggregation + plot)
- Relationship between categorical ↔ categorical features (e.g., crosstab)
- Distribution of a numerical feature (histogram)

c) Examine the extent to which there are features with irregular cardinality in the data. Apply appropriate preprocessing steps to address this issue.

Step1 : Text Normalisation

Normalise categorical text to reduce irregular cardinality

```
for col in cat_cols:
```

```
df_clean[col] = (
    df_clean[col]
    .astype(str)
    .str.lower()
    .str.strip()
)
```

Step 2: Re-check Cardinality (IMPORTANT)

```
df_clean[cat_cols].nunique()
```

This shows whether cardinality has reduced after preprocessing.

~~~~~

### Each category

Hema pg 7

~~~~~

Produce a plot showing how the total sale amount for cars sold in the full dataset relates to the region in which the sale took place.

Plot code (for 2)

Just have to change the grp name ...to find the grp name u have to perform for each category given

~~~~~

**plot( for 1)**

### Distribution (histogram)

#####

### Task 3

Filter the dataset to only include cars sold in either the Leinster or Munster region. Report the percentage of rows retained in the filtered dataset.

```
df_filtered = df_clean[ df_clean["CATEGORICAL_COLUMN"].isin(["VALUE_1",  
"VALUE_2"]) ]
```

Percentage:

```
percentage_retained = (len(df_filtered) / len(df_clean)) * 100
```

```
percentage_retained
```

### Example

>>>>>Monthly time series of total sale amount per product category

```
# Ensure date column is datetime
```

```
df_clean["date"] = pd.to_datetime(df_clean["date"])
```

```
# Create monthly total sales per product category
```

```
monthly_category_sales = (
```

```
    df_clean
```

```
    .groupby([pd.Grouper(key="date", freq="M"), "product_category"])["sale_amount"]
```

```
    .sum()
```

```
    .unstack()
```

```
)
```

```
# Plot all categories on one figure

monthly_category_sales.plot(figsize=(10, 5))

plt.xlabel("Month")

plt.ylabel("Total Sale Amount")

plt.title("Monthly Total Sale Amount by Product Category")

plt.show()
```

>>>>3-month rolling average for each category

```
# Calculate 3-month rolling average

rolling_avg = monthly_category_sales.rolling(window=3).mean()
```

```
# Plot rolling averages

rolling_avg.plot(figsize=(10, 5))

plt.xlabel("Month")

plt.ylabel("3-Month Rolling Average of Sale Amount")

plt.title("3-Month Rolling Average of Monthly Sales by Product Category")

plt.show()
```





Hems

# Python Data Science Practical Exam – Ready-to-Paste Templates

---

## **TEMPLATE 0: Notebook Setup (ALWAYS FIRST CELL)**

```
# === IMPORTS ===  
# Do NOT remove unless explicitly unnecessary  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression, LogisticRegression  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import accuracy_score, mean_squared_error  
  
# Make plots display nicely  
plt.rcParams['figure.figsize'] = (8, 5)
```

 **Change nothing here unless instructed**

---

## **TEMPLATE 1: Data Loading & Initial Inspection**

### **Markdown Cell**

```
## Task X – Data Loading and Inspection  
We begin by loading the dataset provided and inspecting its structure.
```

### **Code Cell**

```
# CHANGE: replace with actual file name provided in exam  
df = pd.read_csv("DATASET_NAME.csv") #CSV  
df = pd.read_csv("data.tsv", sep="\t") # TSV  
df = pd.read_excel("data.xlsx") # Excel  
# Quick inspection  
df.head()
```

```
# Dataset structure and summary  
df.info()
```

```
# Statistical summary (numerical columns only)  
df.describe()
```

```
#Provides min, max, mean, std, and percentiles for numeric columns.  
df.describe(include="all").T
```



**Change:**

- `DATASET_NAME.csv` → actual dataset name
- 



## TEMPLATE 2: Data Cleaning & Preprocessing

### Q5. Identify numeric and categorical features

```
import numpy as np  
  
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()  
  
cat_cols = df.select_dtypes(exclude=[np.number]).columns.tolist()  
  
print("Numeric:", num_cols)  
  
print("Categorical:", cat_cols)
```

### Markdown Cell

```
## Data Cleaning
```

The dataset is cleaned to ensure it is suitable for analysis and modelling.

### Code Cells

```
# Check for missing values
```

```
df.isna().sum()
```

```
# OPTION 1: Drop missing values (use if many rows exist)
```

```
df_clean = df.dropna()
```

```
# OPTION 2: Fill missing values (use if dropping is not suitable)  
# CHANGE: choose appropriate column and strategy  
df_clean = df.copy()  
df_clean['COLUMN_NAME'] =  
df_clean['COLUMN_NAME'].fillna(df_clean['COLUMN_NAME'].mean())
```

```
# Verify cleaning  
df_clean.isna().sum()
```

**TO FILL THE MISSING VALUES (use if dropping is not suitable)**  
**If the column is CATEGORICAL / TEXT**

(brand, name, type, category, city, make, model)

```
Df['column name'] = df['column name'].fillna(df['column  
name'].mode()[0])
```

**If the column is NUMERIC**

(age, price, mileage, salary, score)

```
df['column name'] = df['column name'].fillna(df['column  
name'].median())
```

```
df_clean = df.copy()  
  
cat_cols = ['name', 'name 2', 'name 3']  
  
for col in cat_cols:  
    df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])  
  
df_clean.isna().sum()
```

**#Report the percentage of rows with missing values and remove these rows from the dataset.**

```
missing_percentage = (df.isnull().sum() / len(df)) * 100  
print("Percentage of missing values per column:\n",  
missing_percentage[missing_percentage > 0])
```

*"Report the percentage of rows with missing values and remove these rows from the dataset."*

```
# Identify rows with at least one missing value
rows_with_missing = df.isna().any(axis=1)

# Calculate percentage of rows with missing values
percentage_missing_rows = (rows_with_missing.sum() / len(df)) * 100
percentage_missing_rows

# Remove rows with missing values
df_clean = df.dropna()
```

*Change: sometimes you only drop based on specific columns:*

```
df.dropna(subset=["col1", "col2"])
```

*#Remove duplicates*

```
duplicates = df.duplicated().sum()
print("Duplicates:", duplicates)
df = df.drop_duplicates()
```

#### One-line explanation to write

"Numeric features are imputed using the median, while categorical features are imputed using the mode."

#### Change:

- **COLUMN\_NAME** → column with missing values
- Choose **only one** strategy (drop OR fill)

---

## Q7. Detect outliers (IQR method)

```
Q1 = df[num_cols].quantile(0.25)
Q3 = df[num_cols].quantile(0.75)
IQR = Q3 - Q1
```

```
outliers = ((df[num_cols] < (Q1 - 1.5*IQR)) | (df[num_cols] > (Q3 + 1.5*IQR))).sum()
print(outliers.sort_values(ascending=False))
```

## Q8. Visualise features

```
import matplotlib.pyplot as plt
# Histogram
df["some_numeric_column"].plot(kind="hist", bins=20)
plt.title("Distribution")
```

### *#If you want a cleaned histogram*

```
plt.figure(figsize=(7,4))
plt.hist(df["amount"].dropna(), bins=30)
plt.title("Distribution of product sale amounts")
plt.xlabel("amount (€)")
plt.ylabel("frequency")
plt.tight_layout()
plt.show()
```

### *# Bar chart*

```
df["some_category"].value_counts().plot(kind="bar")
plt.title("Category Counts")
plt.tight_layout()
plt.show()
```

### *# change the sales\_by category to the function we make to plot that*

```
sales_by_category.plot(kind="bar")
plt.title("Number of Sales by Product Category")
plt.xlabel("Number of sales")
plt.ylabel("Product Category")
plt.tight_layout()
plt.show()
```

## Q9. Correlation matrix

```
corr = df[num_cols].corr()
import seaborn as sns
```

```
sns.heatmap(corr, cmap="coolwarm")
```

***(b) Correct code: total sales amount by store location (SUM + bar plot)***

```
import matplotlib.pyplot as plt

# columns from the question
LOC_COL = "store_loc"
AMOUNT_COL = "amount"

# total amount per location
total_by_loc =
df.groupby(LOC_COL)[AMOUNT_COL].sum().sort_values(ascending=False)
display(total_by_loc.to_frame("total_sales_amount"))

# plot
plt.figure(figsize=(8,4))
total_by_loc.plot(kind="bar")
plt.title("Total product sales amount by store location")
plt.xlabel("Store location")
plt.ylabel("Total sales amount (€)")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

✅ What to change tomorrow: only LOC\_COL and AMOUNT\_COL if names differ.

```
# Plot Stacked Bar chart
ct.plot(kind="bar", stacked=True, figsize=(10,5))
plt.title("Product category counts by store location")
plt.xlabel("Store location")
plt.ylabel("Number of sales")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

### 3) Number of product sales for each category and each store location

#### How to think

“Number of sales” = **count of rows** per group.

#### What this code does (important)

- `value_counts()` counts how many times each label appears
- That equals **sales count**, because each row is a sale
- `dropna=False` includes missing category/location values (good practice)

```
# ===== CHANGE THESE BASED ON THE QUESTION =====  
CAT_COL = "name"      # e.g., prod_cat, product_category  
LOC_COL = "name"      # e.g., store_loc, location, store  
  
# 1) Sales count by product category  
sales_by_category =  
df[CAT_COL].value_counts(dropna=False).sort_values(ascending=False)  
print("Number of sales by product category:")  
display(sales_by_category.to_frame("num_sales"))  
  
# 2) Sales count by retail store location  
sales_by_location =  
df[LOC_COL].value_counts(dropna=False).sort_values(ascending=False)  
print("Number of sales by store location:")  
display(sales_by_location.to_frame("num_sales"))
```

---

### 4) Plot: number of sales vs product category (bar chart)

#### How to think



If they ask: “**Create a plot showing how number of sales relates to category**”

→ Bar chart of the counts per category.

```
plt.figure(figsize=(8,4))
sales_by_category.plot(kind="bar")
plt.title(f"Number of product sales by {CAT_COL}")
plt.xlabel(CAT_COL)
plt.ylabel("Number of sales")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

## (c) Relationship between store location and product category

Best answer: a **crosstab** (counts) + optionally a **normalised crosstab** (proportions).

```
LOC_COL = "store_loc"
CAT_COL = "prod_cat"

# counts table
ct = pd.crosstab(df[LOC_COL], df[CAT_COL])
display(ct)

# proportions per location (each row sums to 1)
ct_norm = pd.crosstab(df[LOC_COL], df[CAT_COL], normalize="index")
display(ct_norm)
```

# TIME SERIES ANALYSIS – EXAM NOTES TEMPLATE (Copy/Paste)

## Exam phrase to remember

- “I converted the date column to datetime, aggregated by month, and plotted the series to identify trends and seasonality.”
  - “A rolling average smooths short-term noise and makes the underlying trend clearer.”
- 

## 0) Placeholders (ONLY things you change in exam)

```
# ===== CHANGE THESE BASED ON YOUR DATASET / QUESTION =====  
DATE_COL    = "COLUMN_DATE"          # e.g., sale_date, date,  
transaction_date  
GROUP_COL   = "COLUMN_GROUP"         # e.g., prod_cat, fuel_type,  
category  
VALUE_COL   = "COLUMN_VALUE"         # e.g., amount, sale_amount, price  
(numeric)  
  
# Optional filter (only if asked)  
FILTER_COL  = "COLUMN_FILTER"        # e.g., region, store_loc, location  
KEEP_VALUES = ["VALUE1", "VALUE2"]  # e.g., ["Leinster", "Munster"]
```

---

## 1) Convert date + basic checks (ALWAYS DO FIRST)

 Why: time series needs real datetimes.

```
import pandas as pd  
import matplotlib.pyplot as plt  
  
# Convert date column to datetime (safe)
```

```
df[DATE_COL] = pd.to_datetime(df[DATE_COL], errors="coerce")

# Remove rows where date is missing (common exam requirement)
df_ts = df.dropna(subset=[DATE_COL]).copy()

# Check time range (helps confirm parsing is correct)
print("Date range:", df_ts[DATE_COL].min(), "to",
      df_ts[DATE_COL].max())
print("Rows:", len(df_ts))
```

---

## 2) (COMMON) Monthly series for the whole dataset (count)

### Question style:

**“Create a monthly time series showing number of records/sales per month.”**

```
# Set datetime index (needed for resample)
df_ts = df_ts.set_index(DATE_COL).sort_index()

# Monthly counts (how many rows per month)
monthly_count = df_ts.resample("M").size()

display(monthly_count.head())

plt.figure(figsize=(10,4))
monthly_count.plot()
plt.title("Monthly count of records")
plt.xlabel("Month")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

✓ Change: DATE\_COL only

---

### 3) (COMMON) Monthly totals (SUM) for the whole dataset

Question style:

“Create a monthly time series showing total amount per month.”

```
# Monthly totals (sum of VALUE_COL per month)
monthly_sum = df_ts.resample("M")[VALUE_COL].sum()

display(monthly_sum.head())

plt.figure(figsize=(10,4))
monthly_sum.plot()
plt.title(f"Monthly total {VALUE_COL}")
plt.xlabel("Month")
plt.ylabel(f"Total {VALUE_COL}")
plt.tight_layout()
plt.show()
```

✓ Change: VALUE\_COL

---

### 4) (MOST IMPORTANT) Monthly time series per category (multiple lines) – SUM

## Question style (your exam one):

“For each product category, create a monthly time series of total sale amount per month. Plot together.”

✓ How to think:

- month bucket + group by (month, category) + **SUM amount** + unstack + plot lines

*# Reset index if you want, or use df\_ts with index (either is fine)*

```
df_work = df_ts.reset_index().copy()    # makes grouping easier if
needed
```

```
df_work["month"] =
```

```
df_work[DATE_COL].dt.to_period("M").dt.to_timestamp()
```

```
monthly_total_by_group = (
    df_work.groupby(["month", GROUP_COL])[VALUE_COL]
    .sum()
    .unstack(fill_value=0)
    .sort_index()
)
```

```
display(monthly_total_by_group.head())
```

*# Plot all categories on one figure*

```
plt.figure(figsize=(10,4))
```

```
for g in monthly_total_by_group.columns:
```

```
    plt.plot(monthly_total_by_group.index, monthly_total_by_group[g],
    label=str(g))
```

```
plt.title(f"Monthly total {VALUE_COL} by {GROUP_COL}")
```

```
plt.xlabel("Month")
```

```
plt.ylabel(f"Total {VALUE_COL}")
```

```
plt.legend()
```

```
plt.tight_layout()
```

```
plt.show()
```

✓ Change: GROUP\_COL, VALUE\_COL

## 5) 3-month rolling average per category (multiple lines)

## Question style:

**“From Task 3(a), calculate 3-month rolling average for each category and plot.”**

```
WINDOW = 3 # change if question asks different window

rolling_avg = monthly_total_by_group.rolling(window=WINDOW,
min_periods=1).mean()
display(rolling_avg.head())

plt.figure(figsize=(10,4))
for g in rolling_avg.columns:
    plt.plot(rolling_avg.index, rolling_avg[g], label=str(g))

plt.title(f"{WINDOW}-month rolling average of monthly total
{VALUE_COL} by {GROUP_COL}")
plt.xlabel("Month")
plt.ylabel("Rolling mean")
plt.legend()
plt.tight_layout()
plt.show()
```

✓ Change: WINDOW only (usually 3)

---

## 6) Monthly time series per category – COUNT (if asked “number of sales per month per category”)

```
df_work = df_ts.reset_index().copy()
df_work["month"] =
df_work[DATE_COL].dt.to_period("M").dt.to_timestamp()

monthly_count_by_group = (
    df_work.groupby(["month", GROUP_COL])
        .size()
        .unstack(fill_value=0)
        .sort_index()
)
```

```
display(monthly_count_by_group.head())

plt.figure(figsize=(10,4))
for g in monthly_count_by_group.columns:
    plt.plot(monthly_count_by_group.index, monthly_count_by_group[g],
label=str(g))

plt.title(f"Monthly count by {GROUP_COL}")
plt.xlabel("Month")
plt.ylabel("Count")
plt.legend()
plt.tight_layout()
plt.show()
```

---

## 7) Filtering + % retained (often comes before time series)

**Question style:**

**“Filter to regions A/B and report % retained, then do time series.”**

```
before = len(df_ts)

df_filt = df_ts[df_ts[FILTER_COL].isin(KEEP_VALUES)].copy()

after = len(df_filt)
pct_retained = (after / before) * 100 if before else 0
print(f"Rows retained: {pct_retained:.2f}%")

# Then run the monthly/groupby code using df_filt instead of df_ts
```

✓ Change: FILTER\_COL, KEEP\_VALUES

---

## 8) Mini “trend comment” template (write 2–4 lines)

### Trend summary

- I aggregated the data monthly and plotted one series per category to compare trends over time.
  - The rolling average smooths short-term fluctuations and highlights the underlying trend.
  - The highest category overall appears to be \_\_\_\_\_, with peaks around \_\_\_\_\_.
  - Overall the trend is increasing/decreasing/stable from \_\_\_\_\_ to \_\_\_\_\_.
- 



## Quick cheat rule (memorise)

- “Total sale amount per month” → `sum()`
- “Number of sales per month” → `size()` / `count()`
- “Rolling average” → `.rolling(3).mean()`
- “Multiple series” → `groupby([month, category]).agg(...).unstack()` then plot lines

# Time Series Toolkit Template (COMP47670 style)

## 0) Placeholders you change in the exam

```
# ===== CHANGE THESE BASED ON THE DATASET / QUESTION =====
DATE_COL    = "sale_date"      # e.g., date, sale_date, transaction_date
GROUP_COL   = "prod_cat"       # e.g., prod_cat, fuel_type, category
VALUE_COL   = "amount"         # e.g., amount, sale_amount, price

# Optional (only if asked to filter)
```



```
FILTER_COL = "store_loc"      # e.g., store_loc, region, location
KEEP_VALUES = ["Dublin", "Cork"] # change to values in question
```

---

## 1) Always start like this (parse date + basic checks)

### Why

If date isn't datetime, monthly grouping will fail or give wrong results.

```
import pandas as pd
import matplotlib.pyplot as plt

# Convert to datetime (safe)
df[DATE_COL] = pd.to_datetime(df[DATE_COL], errors="coerce")

# Drop rows where date is missing (common exam step)
df_ts = df.dropna(subset=[DATE_COL]).copy()

# Quick sanity checks
print("Date range:", df_ts[DATE_COL].min(), "to",
      df_ts[DATE_COL].max())
print("Rows:", len(df_ts))
```

---

## 2) Create a “month” column (most common exam requirement)

### Why

They often say “monthly time series” → you need month buckets.

```
# Month bucket (YYYY-MM-01 timestamps)
df_ts["month"] = df_ts[DATE_COL].dt.to_period("M").dt.to_timestamp()
```

## 2) Create a “year bucket” (useful for yearly time series)

This makes a clean timestamp like YYYY-01-01 (similar to your month bucket).

```
df_ts["year_start"] =  
df_ts[DATE_COL].dt.to_period("Y").dt.to_timestamp()
```

### Check it

```
display(df_ts[[DATE_COL, "year_start"]].head())  
df_ts["year_start"].value_counts().sort_index()
```

---

## A) Monthly totals / counts per group (MOST REPEATED)

### A1) Monthly total VALUE per category (Task like “total sale amount per month per category”)

#### What this answers

“For each product category, create a monthly time series showing the **total sale amount** per month... plot together.”

```
monthly_total = (  
    df_ts.groupby(["month", GROUP_COL])[VALUE_COL]  
        .sum()  
        .unstack(fill_value=0)  
        .sort_index()  
)
```

```
display(monthly_total.head())
```

### Plot all categories together (single figure)

```
plt.figure(figsize=(10,4))
for col in monthly_total.columns:
    plt.plot(monthly_total.index, monthly_total[col], label=str(col))

plt.title(f"Monthly total {VALUE_COL} by {GROUP_COL}")
plt.xlabel("Month")
plt.ylabel(f"Total {VALUE_COL}")
plt.legend()
plt.tight_layout()
plt.show()
```

✓ Change tomorrow: DATE\_COL, GROUP\_COL, VALUE\_COL

---

## A2) Monthly count per category (if question says “number of sales per month”)

```
monthly_count = (
    df_ts.groupby(["month", GROUP_COL])
        .size()
        .unstack(fill_value=0)
        .sort_index()
)

display(monthly_count.head())

plt.figure(figsize=(10,4))
for col in monthly_count.columns:
    plt.plot(monthly_count.index, monthly_count[col], label=str(col))

plt.title(f"Monthly number of records by {GROUP_COL}")
plt.xlabel("Month")
```

```
plt.ylabel("Count")
plt.legend()
plt.tight_layout()
plt.show()
```

---

## B) Rolling averages (VERY COMMON)

### B1) 3-month rolling average (Task like “calculate 3 month rolling average”)

#### Key rule

Rolling works correctly only when the index is sorted by time (we already used `.sort_index()`).

```
rolling_3 = monthly_total.rolling(window=3, min_periods=1).mean()
display(rolling_3.head())
```

```
plt.figure(figsize=(10,4))
for col in rolling_3.columns:
    plt.plot(rolling_3.index, rolling_3[col], label=str(col))
```

```
plt.title(f"3-month rolling average of monthly total {VALUE_COL} by {GROUP_COL}")
plt.xlabel("Month")
plt.ylabel("Rolling mean")
plt.legend()
plt.tight_layout()
plt.show()
```

✓ Change tomorrow: same 3 columns only

---

## C) Filtering + % retained (often appears before time series)

### C1) Filter to certain groups (regions/locations) and report % retained

```
before = len(df_ts)

df_filt = df_ts[df_ts[FILTER_COL].isin(KEEP_VALUES)].copy()

after = len(df_filt)
pct_retained = (after / before) * 100 if before else 0

print(f"% rows retained after filtering: {pct_retained:.2f}%")
print("Filtered rows:", after)
```

Then run the same monthly code on `df_filt` instead of `df_ts`.

---

## D) Handling missing months (if plot looks broken / gaps)

Sometimes some months don't exist in the data → lines can look jumpy. You can force all months to appear:

```
# Build a full month index from min to max
full_months = pd.date_range(
    start=monthly_total.index.min(),
    end=monthly_total.index.max(),
    freq="MS" # Month Start
)

monthly_total_full = monthly_total.reindex(full_months, fill_value=0)
```

Then plot `monthly_total_full`.

---

## E) Extra “common asks” (quick marks)

### E1) Which month had the maximum total for each category?

```
max_month = monthly_total.idxmax()
max_value = monthly_total.max()

result = pd.DataFrame({"max_month": max_month, "max_total":
max_value})
display(result)
```

### E2) Month-over-month change (if asked “trend increase/decrease”)

```
mom_change = monthly_total.diff()
display(mom_change.head())
```

### E3) Month-over-month % change

```
mom_pct = monthly_total.pct_change() * 100
display(mom_pct.head())
```

---

## Generic Markdown you can paste under time-series tasks

Use this every time (safe + marks):

```
### What I did
```

- Converted the date column to datetime and created a monthly time index.
- Aggregated the data by month and category to compute the required metric (sum or count).
- Plotted all category series together for comparison.
- Computed a 3-month rolling average to smooth short-term fluctuations.

### ### Why

Monthly aggregation reveals trends more clearly than raw daily records, and rolling averages reduce noise and make patterns easier to interpret.

### ### Key observations

- The highest series overall appears to be \_\_\_\_\_.
- There is an upward/downward/stable trend from \_\_\_\_\_ to \_\_\_\_\_.
- Peaks/troughs occur around \_\_\_\_\_ (possible seasonality or event effects).

---

## What time-series questions can they ask (based on your labs)

You'll usually see one or more of these:

- Monthly **count** per category/location
- Monthly **sum** of amount per category/location
- Plot multiple series on one chart
- **Rolling average** (3-month)
- Filter subset + **% retained**
- Comment trends (highest category, rising/falling, peaks)

### 3(a) Monthly total sale amount per category (single plot)

```
import pandas as pd
import matplotlib.pyplot as plt

# ===== CHANGE THESE IF YOUR COLUMN NAMES DIFFER =====
DATE_COL    = "sale_date"
CAT_COL     = "prod_cat"
AMOUNT_COL  = "amount"

# 1) Convert date column to datetime
df[DATE_COL] = pd.to_datetime(df[DATE_COL], errors="coerce")

# 2) Drop rows where date is missing (so month grouping works)
df_ts = df.dropna(subset=[DATE_COL]).copy()

# 3) Create month bucket (YYYY-MM-01)
df_ts["month"] = df_ts[DATE_COL].dt.to_period("M").dt.to_timestamp()

# 4) Group by month + category and SUM amount
monthly_total = (
    df_ts.groupby(["month", CAT_COL])[AMOUNT_COL]
        .sum()
        .unstack(fill_value=0)
        .sort_index()
)

display(monthly_total.head())

# 5) Plot all categories on one figure
plt.figure(figsize=(10,4))
for c in monthly_total.columns:
    plt.plot(monthly_total.index, monthly_total[c], label=str(c))

plt.title("Monthly total sale amount by product category")
plt.xlabel("Month")
plt.ylabel("Total sale amount (€)")
plt.legend()
plt.tight_layout()
```



```
plt.show()
```

---

### **Task 3(b) 3-month rolling average per category (single plot)**

```
WINDOW = 3 # 3-month rolling average

monthly_roll = monthly_total.rolling(window=WINDOW,
min_periods=1).mean()
display(monthly_roll.head())

plt.figure(figsize=(10,4))
for c in monthly_roll.columns:
    plt.plot(monthly_roll.index, monthly_roll[c], label=str(c))

plt.title("3-month rolling average of monthly total sale amount by
category")
plt.xlabel("Month")
plt.ylabel("Rolling average (€)")
plt.legend()
plt.tight_layout()
plt.show()
```